

# Fan-Out/Fan-In Pattern

 [systemdesignschool.io/fundamentals/fan-out-fan-in](https://systemdesignschool.io/fundamentals/fan-out-fan-in)



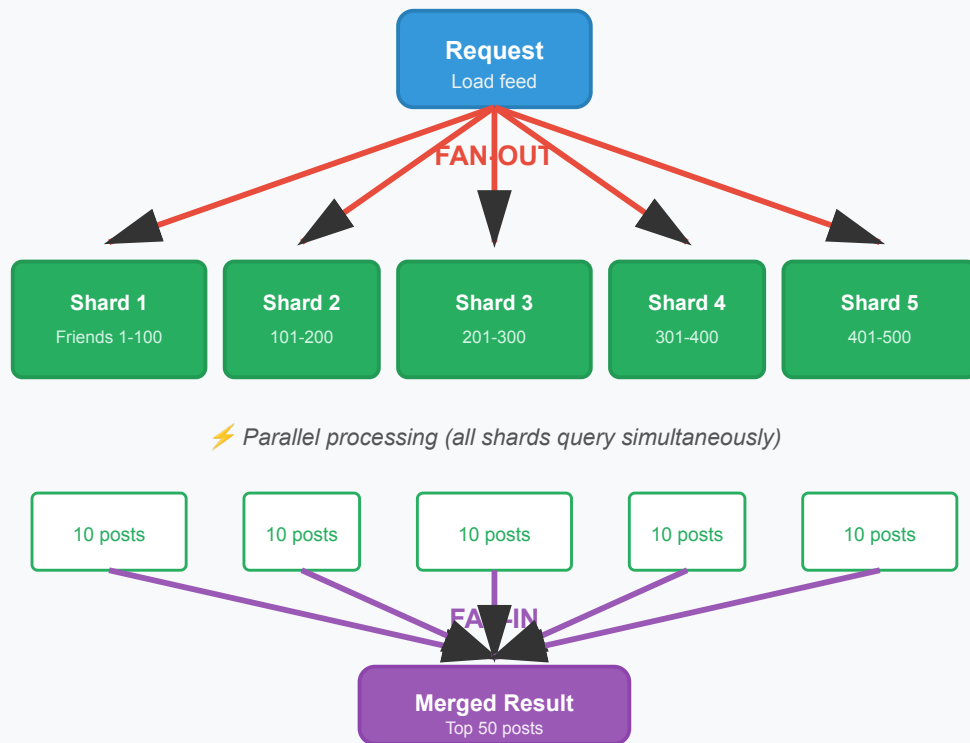
A user opens their news feed. The system must fetch posts from 500 friends. Querying 500 users sequentially takes seconds. The user expects results in milliseconds. Fan-out/fan-in solves this by distributing work across multiple database shards in parallel then aggregating results.

## The Pattern

Fan-out splits a single request into multiple parallel sub-requests. Each sub-request processes independently on different servers or shards. Fan-in collects all sub-request results and combines them into one response.

Twitter timeline generation demonstrates this. When @alice loads her timeline, the system fans out to fetch recent tweets from each person she follows. If Alice follows 300 people, the system queries 300 user timelines. Running these queries in parallel across 10 database shards reduces latency from seconds to milliseconds.

## Fan-Out/Fan-In Pattern



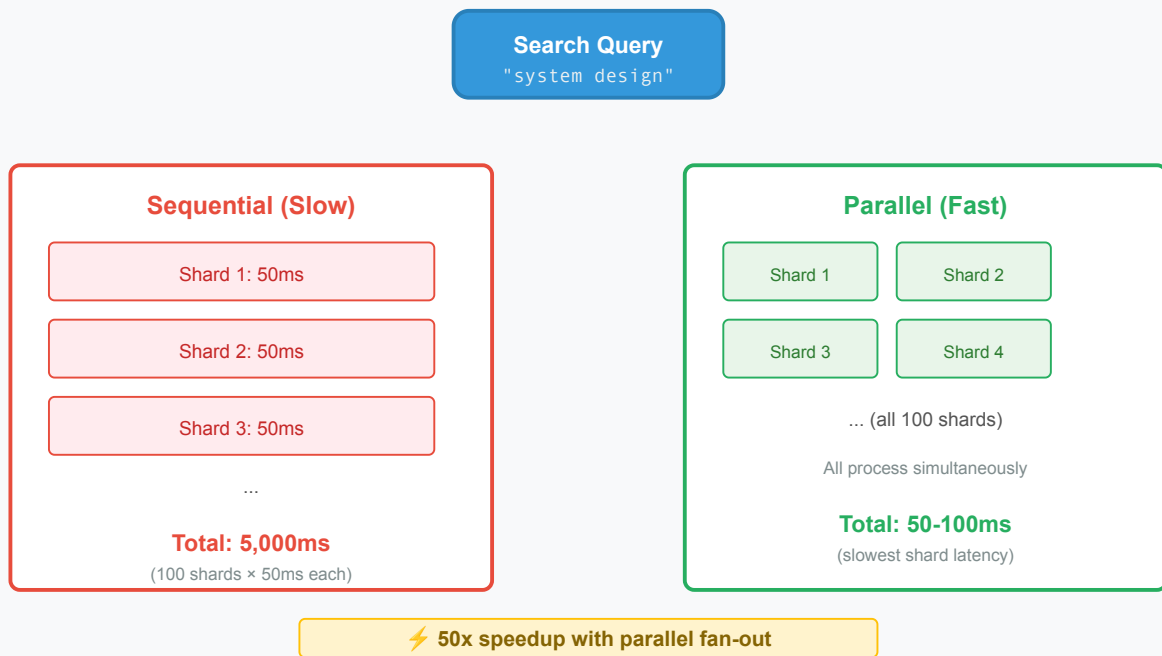
The pattern appears in two scenarios: parallel reads and parallel writes. News feeds fan out reads across shards. Notification systems fan out writes to millions of user inboxes.

### Fan-Out Read Pattern

Search engines use fan-out reads. A query for "system design" searches billions of documents. The search index partitions into 100 shards. The query fans out to all 100 shards. Each shard returns its top 10 matching documents. Fan-in merges the 1,000 results and returns the global top 10.

Sequential processing would query each shard one after another. At 50ms per shard, 100 shards take 5,000ms. Parallel fan-out queries all shards simultaneously. The total latency equals the slowest shard, typically 50-100ms. The 50x speedup makes the search practical.

## Fan-Out Read: Search Query



E-commerce product search follows the same pattern. A query for "laptop under \$1000" searches across product catalog shards. Each shard filters by price and category. Results merge and sort by relevance. The user sees top matches within milliseconds.

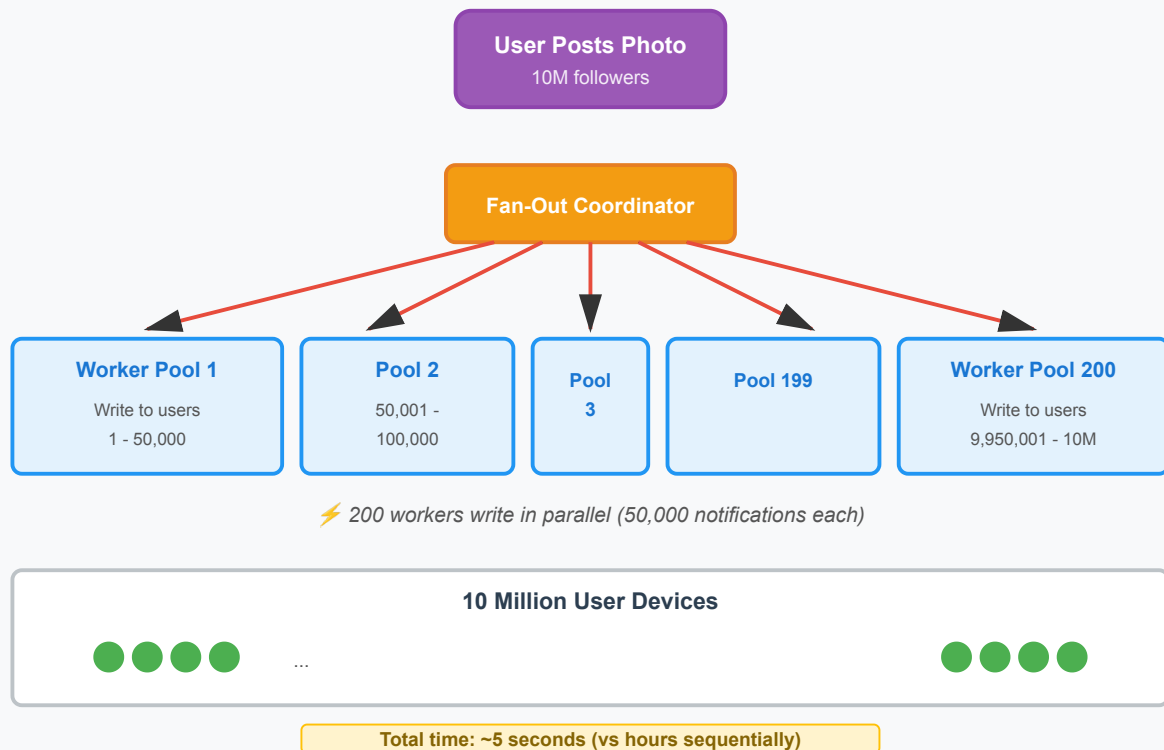
The challenge is handling stragglers. If 99 shards respond in 50ms but one takes 500ms, the entire request waits. Solutions include timeouts with partial results, backup requests to replicas, or hedged requests that query multiple replicas and use the first response.

## Fan-Out Write Pattern

Notification systems use fan-out writes. When a user posts a photo, the system sends notifications to all followers. A celebrity with 10 million followers triggers 10 million notification writes.

Writing 10 million notifications sequentially is impossible within reasonable time. The system fans out writes across hundreds of servers. Each server handles 50,000 notifications. Total write time drops from hours to seconds.

## Fan-Out Write: Notifications



Twitter's timeline architecture uses fan-out writes for the home timeline. When @alice tweets, the system writes her tweet to the inbox of every follower. This pre-computes the timeline. When a follower loads their feed, it reads from their pre-built inbox instead of querying all followees at read time.

The trade-off is write amplification. One tweet generates millions of writes for popular accounts. Twitter uses a hybrid approach. Regular users get fan-out writes. Celebrities with millions of followers skip fan-out. Their followers merge celebrity tweets at read time to avoid billions of writes per tweet.

## Aggregation Strategies

Fan-in must combine results from multiple sources. The aggregation strategy depends on the operation.

Merge sort combines sorted lists. Search results from each shard come pre-sorted by relevance. Fan-in merges them maintaining sort order. This produces globally ranked results efficiently.

Union combines all results without duplicates. A news feed pulls posts from multiple friend groups. Fan-in unions all posts and deduplicates by post ID.

Intersection finds common results. A search for "red laptop" intersects results for "red" and results for "laptop". Only documents in both sets appear in final results.

Aggregation (sum, count, average) combines numeric values. An analytics query counts total orders across regional databases. Each region returns its count. Fan-in sums them for the global total.

## Handling Failures

---

Partial failures are inevitable. When fan-out queries 100 shards, one might fail. The system must decide: wait indefinitely, timeout and fail, or return partial results.

Returning partial results maintains availability. If 99 out of 100 shards respond, return those results with a warning. Search engines do this routinely. One missing shard removes 1% of results but keeps the service available.

Timeouts prevent one slow shard from blocking everything. Set a deadline like 100ms. Shards responding within the deadline contribute results. Slow shards get excluded. This provides bounded latency at the cost of completeness.

Backup requests improve reliability. Send the request to the primary shard. If it doesn't respond within 50ms, send to a replica. Use whichever responds first. This masks failures and slow nodes with minimal extra load.

## Interview Application

---

When asked to design a news feed, explain the fan-out/fan-in pattern explicitly. Alice follows 500 people. Loading her timeline fans out to 500 user timelines across database shards. Each shard returns the 10 most recent posts from its users. Fan-in merges 5,000 posts and selects the 50 most recent.

Discuss the write path alternative. Instagram pre-computes timelines using fan-out writes. When Bob posts, the system writes to all follower inboxes. This moves work from read time to write time. Reads become simple inbox lookups instead of complex aggregations.

Mention the celebrity problem. Accounts with millions of followers create write amplification. One post generates millions of writes. Suggest a hybrid approach: fan-out writes for regular users, fan-in reads for celebrities. Show awareness of the trade-off.

For notification systems, explain fan-out writes to millions of devices. Mention work queues to parallelize the writes. Discuss idempotency in case of retries. Show you understand both the distribution mechanism and the failure scenarios.

This pattern appears in most large-scale system designs. Demonstrating understanding of when to fan-out reads versus writes, how to aggregate, and how to handle failures shows strong distributed systems knowledge.